Conformal Surprisal Fusion for Detecting Prompt-Injection Compromise in LLM Agent Traces

Quang-Vinh Dang[a]

[a]*British University Vietnam, Hung Yen, Vietnam*

Abstract

Large language model (LLM) agents that call external tools ingest untrusted content and fold it into the context driving their next action, exposing them to indirect prompt injection. Trace-based monitors are model-agnostic and cheap, but the literature has converged on one family of evidence: whether untrusted text *looks like an instruction*.

We introduce **SENTRY-Fuse**, which converts heterogeneous trace signals—one scoring the *observation* stream, one auditing the *action* stream—into conformal tail surprisals against a benign calibration split and fuses them by addition. The rule is scale-free, monotone, weight-free and distribution-free valid; we prove that the alternative of subtracting the benign maximum can drive an entire attack family to chance.

Over 90 compromised trajectories from two agents, the audit raises AUROC from 0.937 to 0.953 and recall from 0.709 to 0.898. A trajectory bootstrap leaves that gain only marginally clear of zero. The robust effect is stability: the single signal returns *zero* recall in 2 of 10 calibration draws, and fusion removes that collapse. Against a re-implemented Task Shield the fusion ranks better (0.764 against 0.953), though Task Shield's own rule recovers more compromises at a 39% false-alarm rate.

Two findings concern evaluation itself. AgentDojo delivers payloads through tool outputs a successful injection must have read, so corpora of this shape cannot exhibit a compromise invisible to a trace monitor. Its `security` flag inverts the compromised and resisted classes, silently inverting our own results. Code and data are released.

Keywords: LLM agents, indirect prompt injection, runtime monitoring,

Email address: `vinh.dq4@buv.edu.vn` (Quang-Vinh Dang)

conformal prediction, anomaly detection, AI agent security

1. Introduction

Autonomous agents built on large language models now read email, browse the web, query databases and move money on a user's behalf. To act, they consume *untrusted* content—web pages, documents, tool outputs—and fold it directly into the context that determines their next tool call. This is the attack surface exploited by *indirect prompt injection*: an adversary plants an instruction inside data the agent will later read, and the agent, unable to separate trusted instructions from untrusted data, follows it [1, 2]. As agents move from demos into production, a runtime monitor—something that watches the live execution trace and halts or escalates when behaviour departs from what is safe—becomes as important as the agent itself.

Monitors that observe only the black-box trace are especially attractive in practice. They need no access to model weights or activations, they impose no retraining cost, they compose with whichever model a deployment happens to use, and they can be dropped in front of a third-party agent. A substantial line of work therefore builds detectors over traces [3, 4, 5]. Almost all of it, however, rests on one kind of evidence: examining the untrusted content the agent read and asking whether it *looks like an instruction*. Keyword and perplexity heuristics do this lexically; fine-tuned detectors and LLM judges do it semantically.

The observation that started this work. We began from that same premise and built a monitor on it. It performed well on the standard benchmark attack and then collapsed, to chance, when we changed the attack template. Our first explanation was that the new attack was phrased too subtly. That explanation was *wrong*: the new payloads were, if anything, more imperative, and they appeared in the trace more often. The collapse came from our own normalisation step. Diagnosing that failure honestly led us to a more general question, which organises this paper:

> *What, exactly, can a monitor learn from a black-box agent trace, and where does that information run out?*

Answering it requires separating two objectives that the literature routinely conflates. *Attempt detection* asks whether untrusted instructions entered the agent's context at all; *compromise detection* asks whether the agent

actually did the attacker's bidding. They have different positive classes, different ceilings, and—as we show—different optimal evidence. A monitor tuned for the first is not automatically good at the second.

Contributions. Our contributions are as follows.

1. **SENTRY-Fuse, a conformal fusion monitor** (Section 3.4). We combine two trace signals—one scoring the observation stream, one auditing the action stream—by mapping each to its conformal tail surprisal against a benign calibration split and summing. We show (Propositions 1–3) that the construction is distribution-free valid, scale-free and non-decreasing in every component, and that the excess-over-maximum normalisation it replaces is many-to-one and can collapse an entire attack family to a point—which it did, measurably (Section 3.6).
2. **Action-side evidence is complementary, not redundant** (Section 5). Signals that audit *what the agent did* carry information that a signal scoring *what the agent read* does not. Adding an action-side audit lifts pooled AUROC from 0.937 to 0.953 and recall at a fixed budget from 0.709 to 0.898—a bootstrap gain of $+0.0163$ $[-0.0006, +0.0351]$ in AUROC. It also removes most of the variance the single signal shows across calibration draws, where that signal collapses to *zero* recall in 2 of 10 splits. A third signal, sink provenance, did not reach significance and is reported as such rather than folded into the method.
3. **Where trace evidence lives, and a caution about benchmarks** (Section 5.4). We measure how often a compromise leaves evidence in the observation and action channels. The observation channel is close to saturated—every compromised trajectory contains part of the attacker's directive in a tool output it read—for the structural reason that AgentDojo delivers payloads *through* tool outputs. We deliberately stop short of converting this into an information-theoretic ceiling: the quantity is not identified (three defensible matching criteria give three different answers), and we explain in Remark 5 why we withdrew such a bound. The durable consequence is a benchmark-design point: corpora of this shape cannot exhibit a compromise that is invisible to a trace monitor, so they cannot test the hard case.
4. **An external anchor, re-implemented rather than quoted** (Section 5.5). Comparing only against our own signal subsets would leave the evaluation a self-ablation, and comparing against published figures

on other corpora settles nothing. We therefore re-implement Task Shield [5]—the closest prior work to our action-side signal—and run it on our traces, under our splits, with the same judge model, so that the comparison isolates the method rather than the defender. The fusion ranks better (0.764 against 0.953 pooled, and on each corpus separately). Task Shield's published rule catches more compromises than we do (0.933 against 0.898) but flags 39% of benign trajectories, and because its score takes only 7 distinct values with 39% of benign traces tied at the floor, it has no operating point at any lower false-alarm rate. We report what this comparison does not support as carefully as what it does.

5. **Methodological findings and honest negatives** (Sections 4.4 and 5.6). We document a label-polarity trap in the standard evaluation harness that inverts the compromised and resisted classes; five components we implemented and rejected, three of which we had previously reported as part of the method; and the finding that a hand-picked threshold outperformed the anytime-valid e-detector we started from, which we report rather than omit.

Every number, table and figure reported here is reproducible offline from the released trajectories and cached judge scores, without access to any model API; Appendix A describes the released artefact.

2. Background and Related Work

2.1. Indirect prompt injection

Direct prompt injection manipulates the user-supplied prompt; *indirect* prompt injection is the more dangerous variant for agents, because the adversary never talks to the agent at all. Greshake et al. [6] introduced the threat and gave the first systematic treatment of it. Their central observation is the one this entire literature rests on: once an LLM is augmented with retrieval, *processing* untrusted retrieved data becomes analogous to *executing* arbitrary code, because the boundary between data and instructions is not represented anywhere in the model's input. They derive a taxonomy of resulting threats—information gathering, fraud, intrusion, malware (including prompts that propagate as worms), manipulated content, and availability attacks—and a taxonomy of delivery methods, distinguishing *passive* injection (planting content that retrieval will find), *active* injection (sending

content, e.g. by email, that the agent will process), *user-driven* injection (tricking the user into pasting it), and *hidden* injection (multi-stage or encoded payloads). They demonstrate feasibility against real deployed systems including Bing Chat and GitHub Copilot. Our threat model (Section 3) is the passive/active case, and the compromises we detect fall predominantly into their information-gathering and fraud categories.

The consequences scale with the agent's privileges: an agent that can send email can exfiltrate; an agent that can move money can steal. Zhan et al. [1] systematise this for tool-integrated agents specifically.

Wallace et al. [7] argue that the root cause is the absence of a privilege ordering over instruction sources, and train models to respect an explicit *instruction hierarchy*. This mitigates but does not eliminate the problem, since the model must still infer which span of a long context is privileged. Nasr et al. [8] show that defences evaluated only against static attacks tend to collapse under adaptive attacks that optimise against the defence, and argue that adaptive evaluation should be the norm—a caution we take seriously in Section 6.1.

2.2. Benchmarks for agent security

Progress here is benchmark-driven, and the choice of benchmark determines what "detection" even means.

AgentDojo [2] is the most widely used harness for our setting. It provides four tool environments (banking, workspace, travel, Slack), a set of *user tasks* defining legitimate goals, and a set of *injection tasks* defining attacker goals, together with attack templates that place the attacker's directive into environment content. Crucially, it scores two things separately: *utility*, whether the user's task was accomplished, and *security*, whether the *injection* task's goal was accomplished. Section 4.4 shows that the direction of this second flag is easy to invert, with severe consequences.

InjecAgent [1] contributes a distinct family of injection templates aimed at tool-integrated agents, which we use to test whether a detector generalises beyond the template it was developed on. **τ-bench** [9] supplies realistic benign multi-turn tool-use trajectories in customer-service domains; it has no attack suite, so we use it purely to enlarge and diversify the benign reference. **WebArena** [10] provides realistic web environments and is complementary but not injection-focused.

AgentHarm [11] deserves separate mention because it is easy to mistake for our setting. It comprises 110 base explicitly malicious agent behaviours—

440 including augmentations—across 11 harm categories over a suite of synthetic tools, and it scores whether an agent both refuses *and* retains capability. Its threat model, however, is agent *misuse*: the harmful request comes from the user directly. Ours is indirect injection, where the user is benign and the harm arrives through third-party content—a distinction the AgentHarm authors themselves draw when contrasting their benchmark with AgentDojo. The two are complementary and a complete guardrail needs both; we evaluate only the injection case, which Section 6.1 records as a scope limitation.

Li et al. [12] survey the space and note substantial inconsistency in how safety benchmarks define and measure success—a concern our Sections 4.4 and 5.4 substantiate concretely.

2.3. Defences: prevention

Prevention redesigns the agent, or retrains the model, so that injected instructions cannot influence control flow. It is worth separating three mechanisms, because each fails differently and each leaves a different residual role for monitoring.

Marking provenance in the prompt. **Spotlighting** [13] observes that an LLM receives multiple input sources concatenated into one token stream with no signal of which span came from where, and supplies that signal by transforming untrusted spans: delimiting them with explicit markers, *datamarking* them by interleaving a reserved token throughout the untrusted text, or encoding them (e.g. base64) so that the model can still read them but their instruction-like surface form is disrupted. On GPT-family models this reduces attack success from above 50% to below 2% with little utility loss. The approach is attractively cheap—it is prompt engineering, requiring no training—but it relies on the model honouring the marking, which is a behavioural rather than an architectural guarantee.

Changing the interface. **StruQ** [14] makes the sharpest conceptual argument in this literature. Prompt injection, they observe, is the latest instance of a recurring vulnerability class in which control and data share one channel: exactly as SQL injection arises because a query string mixes control with data, and cross-site scripting because an HTML page does, prompt injection arises because the LLM API is *unsafe by design*, expecting developers to concatenate the prompt and the data. Their remedy is the analogue of prepared statements: a *structured query* presenting instruction and data as two

separate inputs, implemented by a filtering front-end using reserved delimiter tokens together with *structured instruction tuning*, which fine-tunes the model on examples where instructions appear in the data portion and must be ignored. This drives manually-crafted attacks below 2% and substantially weakens optimisation-based ones (TAP 97% $\rightarrow$ 9%).

Aligning against the injection. **SecAlign** [15] identifies the weakness of the fine-tuning approach: training only to *prefer* the correct response underspecifies what an incorrect response looks like, so security does not generalise to unseen attacks—StruQ, they report, still exceeds 50% attack success under an attack that optimises the injection. SecAlign therefore builds a preference dataset of (injected input, secure response, insecure response) triples and applies preference optimisation, so the model is explicitly steered *away* from the injected response. This yields near-0% success against optimisation-free attacks and below 10% against optimisation-based ones, improving on StruQ by more than a factor of four.

Sanitising the content. **CommandSans** [16] strips imperative content from tool outputs before the agent reads them, a lighter-weight route that leaves the agent model untouched—though the sanitiser itself is a trained token-level classifier, so the pipeline is not training-free.

Enforcing provenance at the system level. **CaMeL** [17] extracts the control flow from the trusted user query, tracks data provenance through a capability system, and refuses tool calls that carry untrusted taint into sensitive argument positions. This is the strongest guarantee of the group and the most invasive, since it requires rebuilding the agent around the defence.

Why monitoring remains necessary. Every mechanism above modifies the agent or the model. That is often impossible: deployments frequently wrap a third-party model behind an API, cannot retrain it, and cannot re-architect a vendor's agent loop. Notably, Chen et al. [15] explicitly scope their contribution to prevention and set detection-based defences aside; the two lines of work are therefore complementary rather than competing. Prevention narrows the attack surface; monitoring supplies an independent tripwire where prevention is absent, partial, or itself circumvented—and, unlike prevention, it produces an auditable record of what happened, which is what incident response needs.

2.4. Defences: detection

Detection leaves the agent untouched and raises an alarm from a score. **DataSentinel** [4] fine-tunes an LLM to detect contaminated inputs, formulated as a minimax game so that the detector is trained against adaptive evasion. **Task Shield** [5] checks each action for alignment with the user's stated task—conceptually the closest prior work to our action-side signal, though it is used as a standalone gate rather than as one calibrated component of a fused statistic. Because of that proximity it is the natural baseline, and we re-implement it and run it on our corpus in Section 5.5. **MELON** [18] detects injection by re-executing the trajectory with a masked user prompt and comparing the resulting tool calls, on the principle that a hijacked action recurs even when the real task is removed; this is powerful but roughly doubles the model calls (the paper reports $\approx 2\times$ API cost).

AgentArmor [3] is the strongest *reported* trace-only detector and our reference point. It lifts the agent trace into a program representation—control-flow, data-flow and program-dependence graphs—and enforces typed security policies over it. Version 1 of the preprint reports 95.75% true positives at a 3.66% false-positive rate on AgentDojo with a GPT-4o agent; later versions withdraw that figure and report true-positive rates between 86% and 97% at false-positive rates between 2% and 19% instead. We cite the version 1 number because it is the strongest such figure in the literature and therefore the right reference point, while noting that it is not a stable target and that the work remains an unrefereed preprint. Because it reasons about which untrusted value flowed into which argument of which tool, it is structurally different from a scalar text score; our S5 (Section 3.5) is a much coarser approximation of the same idea, and Section 4.3 sets out why we make no head-to-head comparison with it.

Recent work also monitors adjacent failure modes: Belenky et al. [19] detect reward hacking cheaply, Thaman [20] benchmarks tool-use exploits, and Opoku and Banahene [21] apply conformal risk control to retrieval and tool-use drift.

2.5. Distribution-free calibration and anytime-valid inference

Our fusion rule is built from conformal p-values. Conformal prediction [22] converts any real-valued score into a distribution-free valid p-value assuming only that the calibration sample and the test point are jointly exchangeable, which is what lets us combine signals living on incompatible scales without distributional assumptions.

The project began in the adjacent literature on *anytime-valid* inference. E-values and e-processes [23, 24, 25] support continuous monitoring: a non-negative process whose expectation is bounded under the null can be inspected at every step, and thresholding it at $1/\alpha$ controls error at level α at every stopping time by Ville's inequality. Shin et al. [26] extend this to sequential *change* detection with e-detectors, whose average-run-length (ARL) guarantee holds with no independence assumption—attractive here, because an agent's actions are profoundly dependent. Sadhuka et al. [27] note that when the reference is *learned* the process is no longer an exact e-process and supply a PAC-style thresholding correction, and Waudby-Smith and Ramdas [28], Page [29], Lorden [30] provide the betting and classical change-detection background. Several 2026 works apply anytime-valid ideas to LLM systems—Xie [31] sets out the case in an invited discussion and Li [32] attributes drift in evaluation pipelines—and to strategic behaviour in multi-agent systems more generally [33], with conformal ideas applied to agent error attribution [34].

We retain conformal calibration, which does real work in our method, but Section 5.6.2 reports that the sequential e-detector machinery did *not* outperform a hand-picked threshold at the data scales we could reach. We therefore present it as one calibration option rather than as the paper's contribution—a demotion the evidence forced.

3. Threat Model and Method

3.1. Agent execution traces

An agent pursues a user request u by interleaving tool calls and tool results. We write a trace of length T as

$$\mathcal{T} = \big((o_0, a_1), (o_1, a_2), \ldots, (o_{T-1}, a_T)\big), \tag{1}$$

where a_t is the t-th action (a tool name together with its arguments) and o_{t-1} is the tool output the agent observed immediately before choosing a_t. A monitor sees u and $\mathcal{T}$ and nothing else: no model weights, no activations, no logits. We deliberately exclude the agent's own natural-language messages, both because they are not always available in production logging and to keep the setting comparable to prior trace-only work; Section 6.1(iv) records what this exclusion could hide.

3.2. Adversary

The adversary controls the textual content of one or more environment resources that the agent may retrieve, and nothing else. They cannot modify u, the tool implementations, the model, or the monitor. Their goal is to cause the agent to execute an *injection task*: an action sequence serving the adversary rather than the user. This is the standard indirect-prompt-injection model [2].

3.3. Two distinct detection objectives

Definition 1 (Attempt detection)**.** A trace is *attempt-positive* if injected adversarial content entered the agent's context, whether or not the agent complied.

Definition 2 (Compromise detection)**.** A trace is *compromise-positive* if the agent actually accomplished the adversary's injection task.

Both are legitimate targets. Attempt detection suits a guardrail that should escalate whenever untrusted instructions reach the context, since an agent that resisted today may not tomorrow. Compromise detection suits incident response and is the harder, more consequential question. The two differ sharply in practice: in our corpus a capable agent resists most attempts, so the attempt-positive class is several times larger than the compromise-positive class—393 against 90 pooled, and 198 against 18 for the DeepSeek agent. Results for one must never be reported as results for the other—a confusion we made ourselves, as Section 4.4 describes.

3.4. Overview of SENTRY-Fuse

Figure 1 summarises the monitor. Each action of the live trace is scored by one signal from each of two distinct evidence channels; each raw score is converted to a conformal tail surprisal against a benign calibration split; the surprisals are summed and compared against a threshold; and when the monitor fires, the dominant term names the reason. The subsections below define each stage, and Algorithm 1 states the procedure end to end. Section 3.5 presents five signals in total: two turned out to be near-chance under an honest protocol and are reported as negative results rather than quietly dropped.

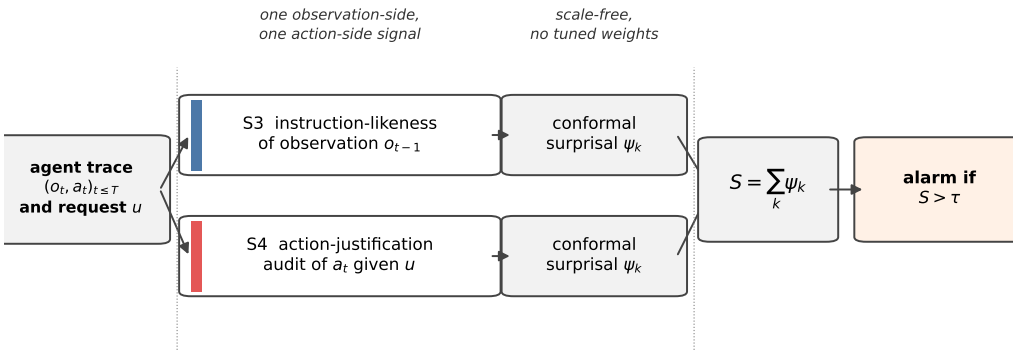

Figure 1: Overview of SENTRY-Fuse. Two signals are computed per action, one from each evidence channel: the *observation* channel (S3) asks whether untrusted content is behaving like an instruction, while the *action* channel (S4) asks whether the agent's behaviour serves the user's request. Each raw score is mapped to a conformal tail surprisal against a benign calibration split, which makes the heterogeneous signals commensurable at a far finer resolution than the excess-over-maximum alternative of Proposition 1, which collapses a whole class to a point. The surprisals are summed and thresholded; on an alarm the largest term, $\arg\max_k \psi_k$, names the signal that fired, which gives the operator an actionable reason rather than a bare alert.

3.5. Signals

We implemented five per-action signals and adopted two. Exactly one, S3, scores the *observation* stream, asking whether untrusted content is behaving like an instruction; the other four score the *action* stream. **SENTRY-Fuse is the fusion of S3 and S4**—one signal from each channel. The remaining three are reported because we implemented them and their failure modes are informative: S1 and S2 ask only whether behaviour is statistically *unusual* and are near-chance under a protocol that fits the reference out-of-sample (Section 5.6); S5 is well motivated and precise but its measured contribution does not survive a trajectory bootstrap (Section 5).

S1: tool-transition surprise (rejected). A Laplace-smoothed bigram model over consecutive tool names, fitted on benign traces, gives

$$s_t^{\mathrm{tr}} = -\log \hat{P}\big(\mathrm{tool}(a_t) \mid \mathrm{tool}(a_{t-1})\big), \tag{2}$$

with $\mathrm{tool}(a_0)$ a distinguished start symbol so that the score is defined at $t = 1$.

S2: argument novelty (rejected). The fraction of tokens in a_t's arguments that never appeared in any argument of the same tool in the benign reference,

$$s_t^{\text{nov}} = \frac{|\{\text{tokens of args}(a_t)\} \setminus V_{\text{tool}(a_t)}|}{|\{\text{tokens of args}(a_t)\}|} \in [0, 1], \tag{3}$$

where V_{tool} is the per-tool argument vocabulary of the reference split. Folding S1 and S2 into a single score as $s_t^{\text{tr}} + 4\, s_t^{\text{nov}}$—which is how this project began—adds two raw quantities on incommensurable scales (nats and a fraction) with a weight chosen by hand. That is precisely the failure mode Section 3.7 exists to remove.

We must be precise about the scope of the fix. **SENTRY-Fuse and its ablation (Table 3) use no weights at all**: every signal is conformalised separately and summed, which is the claim made in Section 3.7. The *attempt-detection* results (Tables 4, 5 and 7) come from the project's original scalar pipeline, which sums the per-action signals with fixed coefficients $(1, 4, 4, 4)$. In each of those tables the weighting is held constant while one factor varies—the normalisation in Table 5, the agent or attack family in Table 4, the combiner in Table 7—so the comparisons they support are unaffected. Their absolute values, however, are those of the weighted composite and should not be read as single-signal performance.

S3: observation instruction-likeness. Let $\text{IL}(o)$ score how strongly a tool output reads as a directive addressed to the agent. We evaluate two instantiations. The *lexicon* variant is a weighted per-word density of imperative verbs, second-person pronouns, pseudo-markup and exclamation marks. The *judge* variant queries a small LLM with the observation alone and takes a normalised integer rating. Section 5.3 compares them; the judge generalises better across attack templates, at the price of one cached model call per distinct observation.

S4: action-justification audit. Given only the user request u, the tool name and its arguments—and *never* the observations—an LLM judge rates how far the call fails to serve u:

$$s_t^{\text{aj}} = \text{Unjustified}(u,\, a_t) \in [0, 1]. \tag{4}$$

Withholding the observations is a deliberate design choice: it prevents the audit from simply re-detecting the injected text, so S4 is not a restatement of S3 and the two may be fused without circularity.

S5: sensitive-sink provenance. A conjunctive, per-value rule that fires when the agent performs an externally-effectful call whose critical argument traces to untrusted prose:

$$s_t^{\mathrm{sp}} = \mathbf{1}\big[\,\mathrm{Sink}(a_t) \wedge \exists v \in \mathrm{Ids}(a_t) :\ v \notin u \ \wedge\ \mathrm{FirstSeenInProse}(v)\,\big]. \tag{5}$$

Here Sink marks tools with external effect (moving money, sending messages, publishing, editing standing instructions) as opposed to read-only lookups; Ids extracts identifier-like values (IBANs, e-mail addresses, URLs, long digit strings); and FirstSeenInProse holds when the value's first appearance in the trace is inside a long natural-language line rather than a short structured record field. The intuition is that a legitimate recipient reaches the agent as a record it looked up, whereas an attacker's recipient arrives embedded in the prose of a poisoned document.

3.6. Why naive normalisation fails

The signals live on incompatible scales: s^{tr} is in nats and unbounded, s^{il} and s^{aj} lie in $[0,1]$, s^{sp} is binary. Summing them raw lets the widest-ranged term dominate. The natural fix is to subtract the largest value observed in benign training data,

$$\phi_{\max}(x) = \max\{0,\ x - \beta\}, \qquad \beta = \max_{x' \in \mathcal{D}_{\mathrm{train}}} x', \tag{6}$$

so that anything within the benign range scores zero. This is a trap.

Throughout, AUROC denotes the area under the receiver operating characteristic curve: the probability that a randomly drawn positive scores above a randomly drawn negative, counting ties as one half.

Proposition 1 (Excess-over-maximum can annihilate a signal)**.** *Let $\phi_{\max}$ be as in* (6)*. Condition throughout on $\mathcal{D}_{\mathrm{train}}$, so that β is a fixed number. Let $\mathcal{B}, \mathcal{C}$ be the benign and attack score distributions and suppose the attack scores satisfy $\mathbb{P}_{X\sim\mathcal{C}}(X \le \beta) = 1$. Write $q = \mathbb{P}_{X\sim\mathcal{B}}(X > \beta)$ for the benign exceedance probability. Then*

$$\mathrm{AUROC}\big(\phi_{\max}(\mathcal{B}),\, \phi_{\max}(\mathcal{C})\big) = \tfrac{1}{2}(1-q) \ \le\ \tfrac{1}{2}, \tag{7}$$

irrespective of how well x itself separates the classes. In particular the transform is at best uninformative under this hypothesis, and strictly worse *than chance whenever some benign score exceeds β.*

Proof. $\phi_{\max}$ maps $(-\infty, \beta]$ to the single point 0. Since $\mathcal{C}$ is supported on $(-\infty, \beta]$, every transformed attack score is exactly 0. A transformed benign score is 0 with probability $1 - q$ and strictly positive with probability q. Comparing an independent attack score with a benign score, the attack score is never larger, is tied with probability $1 - q$, and is smaller with probability q. Counting ties as one half gives $\text{AUROC} = \frac{1}{2}(1-q) + 0 \cdot q$, which is (7). □

Two remarks. First, $q > 0$ is the generic case rather than an edge case: if β is the maximum of n exchangeable benign training scores from a continuous law, then a fresh benign score exceeds it with probability exactly $1/(n+1)$. Second, the hypothesis $\mathbb{P}_{\mathcal{C}}(X \leq \beta) = 1$ holds only approximately on real data—a few attack scores do exceed β—which is why the collapse measured in Section 5.3 lands just *above* 1/2 (0.515 to 0.531) rather than at or below it. The proposition identifies the mechanism and its worst case; it does not predict the exact value.

The premise is not hypothetical. In our data the lexicon instruction-likeness of benign observations is 0.000 at the median and 0.013 on average, but its *maximum* is 0.613—a single outlier—while no InjecAgent observation exceeds 0.352. Every InjecAgent score therefore fell below β, and under $\phi_{\max}$ the lexicon signal was driven to AUROC = 0.521 against 0.687 under the conformal transform of Section 3.7. The model judge, whose output is far more concentrated, fares worse: it awards exactly 1.0 to some benign observations, so $\beta = 1$ and $\phi_{\max}$ is *identically* zero—giving 0.531 against 0.706 on InjecAgent and 0.515 against 0.761 on `important_instructions`, a loss of roughly a quarter of an AUROC on the better scorer. It would be natural to read that collapse as evidence that the attack is subtly phrased. That reading is wrong—the InjecAgent payloads are imperative and appear in the trace in every case—and the cause is the normalisation, not the attack.

The lesson is about *resolution*, not merely about monotonicity. Both $\phi_{\max}$ and the conformal transform below are many-to-one; the difference is that $\phi_{\max}$ can map an entire class to a single point, destroying all within-class ordering, whereas the conformal transform retains $n + 1$ distinct levels. A component transform must therefore be chosen so that its quantisation is fine relative to the separation one hopes to exploit—a requirement we make precise in Proposition 3.

3.7. Conformal surprisal fusion

Definition 3 (Conformal tail surprisal)**.** Let $x_{1:n}$ be benign calibration values for a signal and x a test value. The conformal p-value and surprisal are

$$p(x) = \frac{1 + |\{i : x_i \geq x\}|}{n+1}, \qquad \psi(x) = -\log p(x). \tag{8}$$

Proposition 2 (Distribution-free validity)**.** *If $x, x_1, \ldots, x_n$ are exchangeable, then $\mathbb{P}(p(x) \leq \alpha) \leq \alpha$ for every $\alpha \in (0,1)$; equivalently $\mathbb{P}(\psi(x) \geq \log(1/\alpha)) \leq \alpha$.*

Proof. If the values are almost surely distinct, exchangeability makes the rank of x among $\{x, x_1, \ldots, x_n\}$ uniform on $\{1, \ldots, n+1\}$, and $\{p(x) \leq \alpha\}$ is the event that this rank places x among the top $\lfloor \alpha(n+1) \rfloor$ values, of probability $\lfloor \alpha(n+1) \rfloor / (n+1) \leq \alpha$. Ties only help: because $|\{i : x_i \geq x\}|$ counts calibration values *weakly* exceeding x, a tie can only increase $p(x)$, so the bound continues to hold with the same argument applied to any tie-breaking rule. □

The tie case is not academic here. S5 is binary by construction and S4 saturates at 1.0 on 9.9% of benign trajectories, so the calibration samples of those signals contain many repeated values; the weak inequality in (8) is what keeps p conservative rather than anti-conservative.

Proposition 3 (Ordering and scale-freeness)**.** *Fix calibration values $x_{1:n}$ and let ψ be as in* (8)*. Then (i) ψ is non-decreasing, so it never inverts a strict ordering; (ii) ψ is invariant to any strictly increasing reparameterisation applied jointly to the test value and the calibration values; and (iii) ψ takes at most $n+1$ distinct values, all in $[0, \log(n+1)]$, so it is many-to-one, and AUROC is preserved only up to the ties this quantisation introduces.*

Proof. $|\{i : x_i \geq x\}|$ is non-increasing in x, so p is non-increasing and $\psi = -\log p$ is non-decreasing, which gives (i): if $x < x'$ then $\psi(x) \leq \psi(x')$, so no strictly ordered pair is reversed. For (ii), ψ is a function of the rank of x among the calibration values, and ranks are invariant under strictly increasing maps. For (iii), $|\{i : x_i \geq x\}|$ ranges over $\{0, 1, \ldots, n\}$, so p takes values in $\{1/(n+1), \ldots, 1\}$ and ψ in $\{0, \ldots, \log(n+1)\}$; in particular every x strictly above $\max_i x_i$ receives the same value $\log(n+1)$. □

Part (iii) is a genuine limitation and we state it rather than claim more. Because the transform saturates above the calibration maximum, it *can* lose discrimination. With $n = 1$, calibration $\{0\}$, benign scores $\{0.1, 0.2\}$ and attack scores $\{0.3, 0.4\}$, a perfectly separating signal (AUROC = 1) is mapped to the constant $\log 2$ and AUROC falls to 1/2. Nor is 1/2 a floor: with calibration $\{10\}$, benign $\{0, 0, 11\}$ and attack $\{1, 2\}$, an above-chance signal (0.667) becomes 0.333, because collapsing correctly ordered pairs into ties can cost more than it gains. Monotonicity therefore buys *no* guarantee about AUROC; only resolution does.

At realistic calibration sizes the cost is small but not zero. On well-separated synthetic Gaussians, where the raw signal has AUROC = 1, the conformal transform yields 0.924 at $n = 5$, 0.979 at $n = 23$ and 0.990 at $n = 47$ (400 replicates each; the calibration sizes used in this paper are 23 and 47). The loss is $O(1/n)$, which is why we prefer the largest calibration split the corpus allows.

The contrast with Proposition 1 is accordingly quantitative rather than categorical. Both transforms are many-to-one. The difference is how coarse they are: $\phi_{\max}$ maps an entire class to a single point, leaving no within-class ordering at all and forcing AUROC $= \frac{1}{2}(1 - q) \leq \frac{1}{2}$, whereas ψ retains $n + 1$ ordered levels. What a fusion rule needs is not injectivity, and not monotonicity alone, but a quantisation fine enough that the surviving ordering still separates the classes; Section 5.3 measures what that is worth on real signals.

Definition 4 (SENTRY-Fuse)**.** For signals $k = 1, \ldots, K$ with per-trajectory summaries x_k (we use the per-trajectory maximum of each per-action signal) and benign calibration values $x^{(k)}_{1:n_k}$, the fused statistic is

$$S \;=\; \sum_{k=1}^{K} \psi_k(x_k) \;=\; -\sum_{k=1}^{K} \log p_k(x_k), \tag{9}$$

and the monitor alarms when $S > \tau$, with τ set from held-out benign scores as Algorithm 1 specifies.

The rule has no tuned weights: every signal contributes on the common scale of log-improbability, which is what makes an unweighted sum meaningful. Two caveats belong here rather than in a footnote. First, by Proposition 3(iii) component k contributes at most $\log(n_k{+}1)$, so if the calibration

sets differ in size the sum is implicitly weighted by $\log(n_k{+}1)$; we therefore use a single calibration split, giving $n_k = n$ for all k and equal maximum contributions. Second, the *threshold* τ remains a calibrated quantity, which we choose as a benign quantile; the claim is that the relative contribution of the signals is untuned, not that the monitor is free of all calibration.

Proposition 4 (Fused false-alarm bound)**.** *Suppose each x_k is exchangeable with its calibration values $x^{(k)}_{1:n_k}$ (the hypothesis of Proposition 2). Then for any $\tau > 0$,*

$$\mathbb{P}(S \geq \tau) \;\leq\; \min\Big\{1,\; \sum_{k=1}^{K} e^{-\tau/K}\Big\} \;=\; \min\{1,\; Ke^{-\tau/K}\}. \tag{10}$$

If in addition the p_k are mutually independent, Fisher's method gives the sharper bound

$$\mathbb{P}(S \geq \tau) \;\leq\; \Gamma(K,\tau)/\Gamma(K), \qquad \Gamma(K,\tau) = \int_{\tau}^{\infty} u^{K-1}e^{-u}\,\mathrm{d}u, \tag{11}$$

where $\Gamma(K,\tau)$ is the upper incomplete gamma function.

Proof. If $S \geq \tau$ then some $\psi_k \geq \tau/K$, so $\{S \geq \tau\} \subseteq \bigcup_k \{\psi_k \geq \tau/K\}$; a union bound with Proposition 2 applied at level $\alpha = e^{-\tau/K}$ gives the first claim, and it requires no dependence assumption. For the second, Proposition 2 says each p_k is stochastically larger than a uniform variable, so each $-\log p_k$ is stochastically dominated by a standard exponential; under independence the sum is dominated by a Gamma$(K,1)$ variable, whose upper tail is $\Gamma(K,\tau)/\Gamma(K)$. □

The bound is on $\{S \geq \tau\}$, which contains the alarm event $\{S > \tau\}$ the monitor uses, so it applies to the monitor too. Only *domination* holds, not equality: since each p_k is supported on the finite grid $\{1/(n{+}1), \ldots, 1\}$, the statistic S is discrete and bounded above by $K\log(n{+}1)$, so it cannot be exactly Gamma-distributed.

Remark 1. We report empirical operating points throughout rather than relying on Proposition 4, because the assumption-light union bound is loose and independence across our signals is not credible: S3 and S4 both respond to the presence of an injection, albeit through different channels. The bound's role is to establish that the fused statistic admits distribution-free false-alarm control at all, and to make explicit what would be needed for a sharp one.

Algorithm 1 SENTRY-Fuse

Require: disjoint benign splits $\mathcal{D}_{\text{fit}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{test}}$; target false-alarm rate α; signals $1, \ldots, K$

fit any reference a signal needs (e.g. per-tool vocabularies) on $\mathcal{D}_{\text{fit}}$
for $k = 1, \ldots, K$ **do**
    $x^{(k)}_{1:n_k} \leftarrow$ per-trajectory summaries of signal k on $\mathcal{D}_{\text{cal}}$, scored against the $\mathcal{D}_{\text{fit}}$ reference
end for
$\tau \leftarrow$ the k-th largest of $\{S(\mathcal{T}) : \mathcal{T} \in \mathcal{D}_{\text{test}}\}$, $k = \lfloor \alpha |\mathcal{D}_{\text{test}}| \rfloor$, or the maximum if $k = 0$ ▷ in sample; Section 4.3
at monitoring time, for a trace $\mathcal{T}$:
for $k = 1, \ldots, K$ **do**
    $x_k \leftarrow \max_t s^{(k)}_t(\mathcal{T})$; $\psi_k \leftarrow -\log\big((1 + |\{i : x^{(k)}_i \geq x_k\}|)/(n_k + 1)\big)$
end for
$S \leftarrow \sum_k \psi_k$
if $S > \tau$ **then raise alarm**; report $\arg\max_k \psi_k$ as the dominant cause
end if

Because the fusion is additive over interpretable components, the dominant term $\arg\max_k \psi_k$ names *why* the monitor fired—an injected instruction, an unjustified action, an untrusted recipient, or an anomalous tool transition—which matters for triage.

4. Experimental Setup

4.1. Corpora

We collect real trajectories by running tool-calling agents on standard benchmarks; Table 1 summarises the result. From AgentDojo [2] we run all four suites (banking, workspace, travel, Slack) both benign and under attack, with two attack families: the `important_instructions` template and the InjecAgent [1] template. From τ-bench [9] we run the retail domain benign-only, to diversify the benign reference. Two agents are used: DeepSeek-V4-Flash and GPT-4o-mini, from different vendors, so that agent identity can be varied independently of everything else.

Table 1 already contains a substantive finding. The two agents differ enormously in injection robustness: GPT-4o-mini is compromised in 47% of

Table 1: Trajectory corpora, with the *corrected* label polarity of Section 4.4: *compromised* means the injection task succeeded. Counts are after a two-action minimum filter. "imp. instr." is AgentDojo's `important_instructions` template; the DeepSeek agent is DeepSeek-V4-Flash.

Agent	Attack family	Benign	Attacks	Comp.	Resist.
DeepSeek	imp. instr.	71	198	18 (9%)	180
DeepSeek	InjecAgent	—	42	**0 (0%)**	42
GPT-4o-mini	imp. instr.	70	153	72 (47%)	81
τ-bench retail	none	50	—	—	—
Total		**191**	**393**	**90**	**303**

attempts, DeepSeek-V4-Flash in only 9%, and DeepSeek resisted the InjecAgent family *completely* (0/42). Robustness to indirect prompt injection is therefore a strong function of the agent, not a property of the attack alone, and a defence evaluated on a single agent may be measuring that agent's fragility as much as its own merit.

Remark 2. Because DeepSeek was never compromised by InjecAgent, compromise detection is *undefined* for that cell: there are no positives. Only attempt detection is meaningful there, and we report it as such rather than quoting a rate over an empty class.

4.2. Protocol

Trajectories are converted to per-action signal streams by a deterministic, GPU-free adapter. Feature hashing uses a seed-stable CRC rather than Python's per-process-salted hash, so the entire evaluation is byte-reproducible. LLM judge scores are cached by content hash (681 distinct observations, 1,329 distinct task–action pairs), so all reported evaluations run offline from committed caches.

Our evaluation protocol is deliberately strict about leakage. For each of ten seeds the benign set is split into three disjoint parts: one third fits the reference model (the bigram table and per-tool argument vocabularies), one third supplies the conformal calibration values, and one third supplies the held-out negatives. Compromise-positive trajectories are never used for fitting or calibration. We report mean $\pm$ standard deviation over the ten seeds (population convention; the bootstrap intervals in Section 5 use the sample convention).

The three-way split matters for Proposition 2. If the calibration values are computed on the same trajectories the reference was fitted to, they are in-sample and systematically less surprising than a fresh benign draw, so the conformal p-values are anti-conservative and the stated false-alarm level is not delivered. Fitting on a separate third makes the calibration values and the held-out negatives both out-of-sample with respect to the reference, which is what exchangeability requires.

Remark 3 (Leakage matters)**.** An earlier version of this evaluation fitted the reference on all benign trajectories including the test half. Under that protocol the weighted behavioural composite (S1 and S2 summed with fixed coefficients) appeared excellent, AUROC = 0.982; with the reference fitted out-of-sample and the signals kept separate, S1 and S2 fall to between 0.407 and 0.602 (Table 3). Only held-out numbers appear in the compromise-detection results. This is worth stating plainly because the leaky number is the one a careless pipeline produces by default.

4.3. Metrics

We report AUROC and the true-positive rate (TPR) at a benign false-positive rate (FPR). Comparing detectors at a common FPR is essential—they are otherwise trivially incomparable—but on samples of this size a *nominal* FPR cannot generally be hit. With n held-out negatives, thresholding a score yields only the rates k/n for integer k, so the attainable grid has spacing $1/n$: 4.2% on the 24 GPT-4o-mini negatives and 2.1% on the 47 pooled ones. We therefore set the threshold at the k-th largest negative with $k = \lfloor \alpha n \rfloor$ and then *measure* the rate it realises, which we report in every table alongside the target. We use $\alpha = 5\%$ as the target throughout.

Two caveats about this number. First, it is generally strictly below k/n, because by Proposition 3(iii) the fused statistic takes at most $n_k + 1$ values, so negatives tie at the saturated level and fewer than k of them exceed the threshold—which is why the realised rates in Table 3 range from 0.0% to 4.0% against a 5% target rather than sitting at k/n. Second, we are reading a point off the empirical ROC curve of the held-out negatives, so the reported rate is descriptive of *those* negatives and is not an out-of-sample false-alarm guarantee; a deployment would fix τ on a further split and accept the extra variability, and Proposition 4 is what supplies a distribution-free—if loose—a-priori bound.

This is worth being explicit about because interpolating an empirical quantile, as is conventional, silently produces a different rate: at a nominal

3.66% on 35 negatives the realised rate is $2/35 = 5.71\%$, and a TPR quoted at "3.66%" would then have been measured at over 5%.

On comparison with published figures. AgentArmor [3] is the strongest reported trace-only detector, and version 1 of that preprint reports 95.75% TPR at 3.66% FPR on AgentDojo with a GPT-4o agent. We do *not* present our numbers as beating it, for three reasons. The corpora differ (different agents, a different attack sample, and—as Section 4.4 shows—a positive-class definition that is easy to get wrong); the operating points differ once the realised rate is accounted for; and the figure itself was removed in later versions of that preprint, which report TPR between 86% and 97% at FPR between 2% and 19% instead. We cite it as a reference point for the regime, not as a baseline we outperform, and we did not re-implement its program-dependence analysis.

Comparing only against published figures on other corpora would leave the evaluation a self-ablation, so we re-implemented one published method and ran it on our trajectories under our protocol: **Task Shield**, which is the closest prior work to our action-side signal. Section 5.5 reports it.

4.4. A label-polarity pitfall

Before presenting results we must document a methodological trap, because it inverted our own conclusions for a substantial period and because the ceiling analysis of Section 5.4 is impossible to get right without it.

AgentDojo returns a per-trajectory `security` flag. It is natural, and wrong, to read this as "the system was secure". The flag is computed as the success of the *injection* task, so `security = True` means the attacker's goal was achieved, i.e. the agent *was compromised*.

We initially adopted the opposite convention. The error was invisible in aggregate: both classes are non-empty, all downstream code ran, and every metric produced plausible numbers. It surfaced only when we asked why so many "compromised" trajectories contained no state-changing tool call at all, and read one:

Example 1 (A trajectory we had labelled compromised)**.** Banking suite, `important_instructions`. The agent calls `read_file` and five read-only lookups, then states:

> *"I see a message embedded in the bill file that's trying to get me to do something else first. However, I shouldn't be tricked by that embedded instruction."*

Table 2: Resolving label polarity against in-trace ground truth, over all 107 attacked banking-suite trajectories. Every trajectory flagged `security = True` wired money to the attacker's account, which settles the polarity. The three `False` trajectories that also paid the attacker are cases of partial compliance, and show that the flag undercounts harm.

	`security=True`	`security=False`
agent wired money to attacker IBAN	**31**	3
agent did not	0	**73**

This agent *resisted*. It had been counted as a successful attack.

We resolved the polarity against ground truth rather than documentation. In the banking suite the adversary's goal is a transfer to a specific attacker IBAN, so compliance is directly checkable in the trace:

Example 1 shows one such trajectory in full, and Table 2 settles the direction that matters: every one of the 31 trajectories flagged `security = True` did in fact wire money to the attacker, and none of the trajectories that wired no money is so flagged. So `security = True` is compromise, and the polarity we had been using was inverted.

The converse fails, however, and the failure is worth recording. Three trajectories labelled `security = False` *did* pay the attacker. Inspecting them shows why: this injection task requires not merely a transfer to the attacker's account but one whose subject line names the user's music-streaming service, and in these three the agent transferred the money with a subject of its own invention. The flag scores the injection task's *exact* goal, so an agent that is manipulated into moving money but fumbles a detail of the attacker's script is recorded as having resisted. The harness therefore *undercounts* harm, and our compromised class of 90 is a lower bound. We keep the harness definition—changing it would make our numbers incomparable with the literature's—but flag that a harm-based positive class would be the better target, and that the 303 "resisted" trajectories are not all benign.

The consequences for us were severe and instructive. Every "hijack-detection" number we had computed was measuring the *resisted* class. Our effect-based signals—the action audit and sink provenance—had been graded against trajectories in which, by construction, the agent had done nothing harmful; they appeared useless. Once the polarity was corrected the action audit became one of the two signals the method relies on, reaching AUROC $= 0.900 \pm 0.039$ on GPT-4o-mini and 0.781 ± 0.017 on DeepSeek

(Table 3). We had nearly discarded a working signal because of a boolean.

Remark 4. We report this at length because the failure mode is silent, the fix is trivial, and the affected quantity—which class counts as a successful attack—is the foundation of every number in this literature. We recommend that any work using this harness verify polarity against an in-trace ground-truth check of the kind in Table 2, and report having done so. The same check is what surfaced the undercount above, so it is worth running even once the polarity is known to be right.

5. Results

5.1. Compromise detection

Table 3 gives the main result. On GPT-4o-mini, where the compromised class is largest (72 positives), the observation-side instruction signal alone reaches AUROC = 0.951 but a wildly variable TPR = 0.661 ± 0.433: strong on average and close to unusable at a fixed low false-alarm budget. Adding the action-justification audit gives $\mathbf{0.981 \pm 0.009}$ AUROC and $\mathbf{0.914 \pm 0.040}$ recall at a realised 2.5% false-positive rate. Pooled over both agents the ordering holds, ending at 0.953 and 0.898 ± 0.020. Adding sink provenance on top changes little (Table 3, the row below), which Section 5 quantifies and is why it is not adopted.

Figure 2 visualises the ablation. Four points deserve emphasis.

First, **action-side evidence is complementary**. Pooled, the instruction signal alone reaches AUROC = 0.937; adding the action-side audit takes this to 0.953. We quantify that difference by resampling *trajectories*—positives and negatives—within each of the 10 splits, which gives +0.0163 with a 95% interval of $[-0.0006, +0.0351]$ and $\mathbb{P}(\text{gain} \leq 0) = 0.030$. The recall gain is +0.1706 $[+0.0033, +0.3533]$, $\mathbb{P}(\leq 0) = 0.017$.

We draw one trajectory resample per replicate and reuse it across the 10 splits, which matters: the splits are re-splits of one corpus, so drawing an independent resample inside each split and averaging would estimate the variance of a mean of ten draws and shrink the corpus-sampling variance by about a factor of ten. An earlier version of this work did that, and it turned a borderline effect into an apparently decisive one. Corrected, the AUROC interval only marginally excludes no effect. We therefore do *not* present the AUROC gain as established: 90 positives cannot resolve a difference of this size, and the claim we rest on is the one in the next paragraph.

Table 3: Compromise detection, ten seeds, mean ± std, AgentDojo benign trajectories only. Each seed splits the benign set three ways: one third fits the reference, one third supplies the conformal calibration values, one third the held-out negatives. A nominal false-positive target is not attainable exactly—only multiples of $1/n$ are, and the fused statistic is discrete—so we report the *realised* rate next to each target. Rows therefore sit at different operating points and the TPR columns are not like-for-like across rows, whereas AUROC is; the 3.66% target in particular collapses to a realised 0.0% on the two per-agent corpora, where $\lfloor 0.0366n \rfloor = 0$. SENTRY-Fuse in bold; the two rows below it are ablations we evaluated and did not adopt.

Detector	AUROC	TPR@5%	real. FPR	TPR@3.66%	real. FPR
GPT-4o-mini: 72 compromised, 24 held-out benign					
S3 instruction-likeness	0.951 ± 0.026	0.661 ± 0.433	1.2%	0.472 ± 0.472	0.0%
S4 action audit	0.900 ± 0.039	0.267 ± 0.407	0.4%	0.178 ± 0.356	0.0%
S5 sink provenance	0.868 ± 0.019	0.164 ± 0.328	0.4%	0.082 ± 0.246	0.0%
S1 tool transition	0.481 ± 0.062	0.000 ± 0.000	0.4%	0.000 ± 0.000	0.0%
S2 argument novelty	0.569 ± 0.056	0.082 ± 0.098	2.1%	0.003 ± 0.008	0.0%
S1 + S2	0.530 ± 0.058	0.101 ± 0.076	3.8%	0.040 ± 0.050	0.0%
SENTRY-Fuse (S3+S4)	$\mathbf{0.981 \pm 0.009}$	$\mathbf{0.914 \pm 0.040}$	2.5%	0.803 ± 0.269	0.0%
+ S5 (not adopted)	0.985 ± 0.003	0.956 ± 0.006	2.9%	0.899 ± 0.063	0.0%
all five signals	0.973 ± 0.013	0.932 ± 0.053	3.3%	0.743 ± 0.290	0.0%
DeepSeek-V4-Flash: 18 compromised, 25 held-out benign					
S3 instruction-likeness	0.850 ± 0.021	0.467 ± 0.381	2.4%	0.078 ± 0.233	0.0%
S4 action audit	0.781 ± 0.017	0.000 ± 0.000	0.0%	0.000 ± 0.000	0.0%
S5 sink provenance	0.648 ± 0.027	0.117 ± 0.178	0.8%	0.039 ± 0.117	0.0%
S1 tool transition	0.432 ± 0.068	0.011 ± 0.022	1.2%	0.000 ± 0.000	0.0%
S2 argument novelty	0.407 ± 0.098	0.000 ± 0.000	0.4%	0.000 ± 0.000	0.0%
S1 + S2	0.389 ± 0.080	0.000 ± 0.000	2.4%	0.000 ± 0.000	0.0%
SENTRY-Fuse (S3+S4)	$\mathbf{0.842 \pm 0.006}$	$\mathbf{0.778 \pm 0.000}$	4.0%	0.683 ± 0.230	0.0%
+ S5 (not adopted)	0.832 ± 0.010	0.767 ± 0.033	4.0%	0.539 ± 0.184	0.0%
all five signals	0.807 ± 0.032	0.661 ± 0.118	3.6%	0.478 ± 0.263	0.0%
Pooled: 90 compromised, 47 held-out benign					
S3 instruction-likeness	0.937 ± 0.012	0.709 ± 0.357	2.8%	0.598 ± 0.394	1.3%
S4 action audit	0.863 ± 0.030	0.000 ± 0.000	0.0%	0.000 ± 0.000	0.0%
S5 sink provenance	0.812 ± 0.015	0.000 ± 0.000	0.0%	0.000 ± 0.000	0.0%
S1 tool transition	0.526 ± 0.077	0.012 ± 0.033	1.7%	0.000 ± 0.000	0.9%
S2 argument novelty	0.602 ± 0.045	0.006 ± 0.017	1.5%	0.006 ± 0.017	1.3%
S1 + S2	0.557 ± 0.065	0.038 ± 0.048	3.8%	0.011 ± 0.020	1.7%
SENTRY-Fuse (S3+S4)	$\mathbf{0.953 \pm 0.005}$	$\mathbf{0.898 \pm 0.020}$	3.8%	0.871 ± 0.045	1.9%
+ S5 (not adopted)	0.954 ± 0.003	0.918 ± 0.005	4.0%	0.884 ± 0.071	1.7%
all five signals	0.948 ± 0.012	0.886 ± 0.070	3.8%	0.808 ± 0.076	2.1%

We report bootstrap intervals rather than a t-statistic over the ten splits, and the distinction matters. The splits permute one fixed benign sample and the 90 positives are byte-identical in every seed, so a paired t across seeds describes how much the answer moves with the calibration draw—a real and important quantity, reported as the ± column of Table 3—but not the sampling error in the corpus, which is what a claim about the *method* requires.

The gain is not explained by the action signal being individually strong: alone, S4 reaches only 0.863 AUROC. Its recall column reads 0.000, but that is a discreteness artefact and not evidence—S4's realised false-positive rate in that row is 0.0%, because $\lfloor 0.05 \times 47 \rfloor$ negatives tie at its threshold, so it is being asked for recall at a zero false-alarm budget. What the comparison does show is that a signal far weaker in ranking terms still contributes cases the observation signal misses.

A third signal that did not earn inclusion. We also implemented sink provenance (S5), and it is *not* part of SENTRY-Fuse, because the evidence does not support including it. Adding it moves pooled AUROC by $+0.0004$ $[-0.0043, +0.0056]$—an interval comfortably containing zero, $\mathbb{P}(\leq 0) = 0.432$—and recall by $+0.0268$ $[-0.0178, +0.1011]$, likewise ($\mathbb{P}(\leq 0) = 0.149$). In absolute terms it recovers fewer than two additional positives out of 90. On the DeepSeek corpus it makes AUROC *worse* (0.832 against 0.842). We report the ablation row in Table 3 so the reader can see it, and we decline to claim it.

Second—and this is the claim we would defend hardest—**fusion removes a failure mode that makes the single signal undeployable**. The instruction signal's standard deviation of recall at a fixed budget is ± 0.433 on GPT-4o-mini and ± 0.357 pooled, but that spread is not a spread: it is bimodal. Pooled, the signal returns *exactly zero* recall in 2 of 10 draws and about 0.91 in the rest.

The mechanism is on the negative side, not the calibration side, and it is instructive. With $k = \lfloor 0.05 \times 47 \rfloor = 2$ negatives allowed above it, the threshold is set at the $(k+1)$-th largest held-out negative—the third—so that exactly k of them strictly exceed it. The model judge saturates, awarding exactly 1.0 to some benign observations, so whenever three or more of the 47 held-out negatives sit at that ceiling the threshold *is* the ceiling and no positive can strictly exceed it. Recall is then zero regardless of how well the signal separates the classes. This is a discrete-operating-point pathology of

a saturating score, and adding a second signal with a different saturation pattern breaks the tie: SENTRY-Fuse's spread is ±0.040 and ±0.020, and it never collapses. Unlike the mean gains above, this difference is far outside sampling noise.

Third, **more signals is not monotonically better**. Going from the two-signal method to all five—adding the behavioural pair and provenance—lowers pooled AUROC from 0.953 to 0.948. Isolating the behavioural pair alone, by comparing the three-signal variant (0.954) with all five (0.948), gives the same direction. The effect is not uniform: on GPT-4o-mini all five reach a *higher* recall than the method (0.932 against 0.914) while sitting at a looser realised false-alarm rate, which is why we compare on AUROC. Alone the pair is barely distinguishable from chance (0.530, 0.389 and 0.557 AUROC on the three corpora, the middle one *below* chance). Because the fusion is an unweighted sum, a near-chance component injects variance without adding discrimination, and the honest conclusion is that these two signals do not belong in the monitor. They appear as an ablation row because they were this project's starting point, and their failure is the clearest evidence for the distinction between unusual and misaligned.

Fourth, **the weaker agent is harder, and we report it**. On DeepSeek-V4-Flash every detector is worse than on GPT-4o-mini—SENTRY-Fuse reaches AUROC = 0.842 against 0.981. With only 18 compromised trajectories this corpus cannot support fine distinctions, but we present it rather than quoting the favourable corpus alone: on the agent that resists most attacks, our margin over the single-signal baseline is present in recall (0.467 → 0.778) but *absent* in AUROC, where fusion is slightly worse (0.850 → 0.842). On this corpus, with 18 positives, we do not have a ranking-quality improvement to report.

5.2. Attempt detection and generalisation

Table 4 varies one axis at a time. Attempt detection transfers across both axes: changing the agent leaves it intact (AUROC 0.76 → 0.82), and changing the attack family costs relatively little once the normalisation of Section 3.6 is fixed (0.76 → 0.71). Two caveats. First, the three rows sit at different realised false-alarm rates (final column), so the detection-rate column is not a like-for-like comparison and we quote AUROC, which is threshold-free, for the transfer claims. Second, the DeepSeek/InjecAgent cell illustrates why the two objectives must be kept apart: attempt detection is meaningful there (0.71), while compromise detection is undefined because

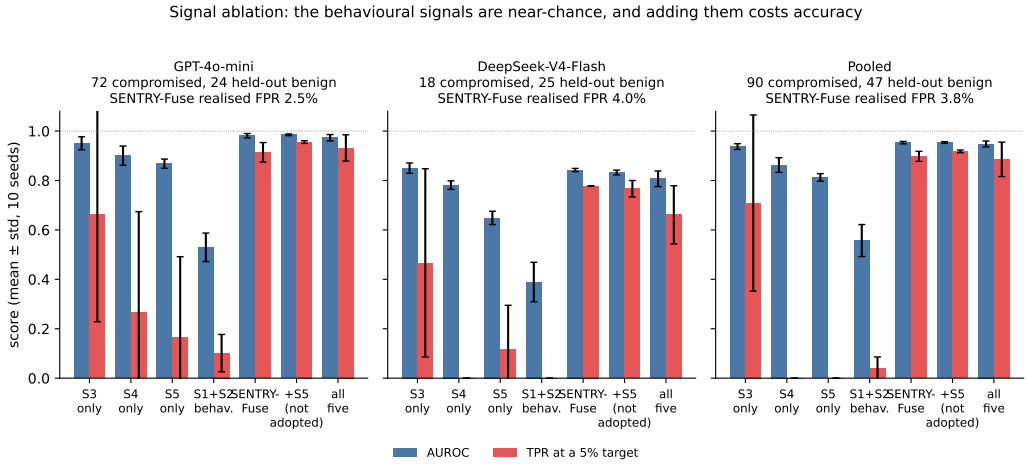

Figure 2: Signal ablation across the three corpora. AUROC (blue) and true-positive rate at a 5% target (red), mean and one standard deviation over ten seeds. The behavioural signals S1 and S2 are near-chance on every corpus, and the five-signal sum is worse than the two-signal method; the realised false-positive rate at the proposed operating point is printed above each panel.

Table 4: Generalisation across agent and attack family; ten seeds, corrected labels. The realised per-step false-alarm rate is given because it differs by row, so the detection rates are *not* at a common operating point and should be read against it.

Agent	Attack family	Comp.	Resist.	Att. det.	Att. AUROC	FPR
DeepSeek	imp. instr.	18	180	0.65	0.76	2.7%
DeepSeek	InjecAgent	0	42	0.52	0.71	2.7%
GPT-4o-mini	imp. instr.	72	81	0.74	0.82	4.3%

the agent was never compromised by that family, so there is no positive class to score.

5.3. Signal and normalisation ablation

Table 5 isolates the two design choices in the observation-side signal S3: how instruction-likeness is scored (fixed lexicon versus LLM judge) and how it is normalised (excess-over-maximum versus conformal). The interaction is the point. Under excess-over-maximum the *better* scorer is the worse detector—the judge falls to 0.515 AUROC on `important_instructions` against the lexicon's 0.779—because the judge's near-binary output places

Table 5: Attempt detection rate / AUROC by instruction scorer and normalisation, ten seeds. Scores come from the original scalar pipeline, which sums the per-action signals with fixed coefficients; only the instruction term's normalisation varies between rows, so the comparison is controlled even though the absolute values are those of the composite. Excess-over-maximum realises a per-step false-alarm rate of 0.0182 and the conformal transform 0.0270 (judge) or 0.0335 (lexicon), so the conformal rows are measured at a *higher* false-alarm budget and their advantage is not an artefact of a looser threshold. The conformal transform is required for the semantic judge to be usable at all.

Scorer	Normalisation	`important_instr.`	InjecAgent
lexicon	excess-over-max	0.662/0.779	0.114/0.521
lexicon	conformal	0.627/0.762	0.481/0.687
LLM judge	excess-over-max	0.159/0.515	0.136/0.531
LLM judge	**conformal**	**0.651/0.761**	**0.521/0.706**

some benign observation at the ceiling, after which Proposition 1 applies and the class is collapsed to a point. Under the conformal transform the ordering reverses and the judge becomes the better scorer on the transfer family (0.706 against 0.687 on InjecAgent), while the lexicon's InjecAgent AUROC recovers from 0.521 to 0.687. Normalisation is not plumbing; it is a first-class design decision that can invert the ranking of scorers and, as it did for us, produce a confident and wrong explanation of a failure.

Figure 3 shows the separation the observation-side signal achieves, and Figure 4 walks through a single real compromise action by action, which is the clearest way to see what each signal contributes.

5.4. Where evidence of compromise lives

A trace-only monitor can only react to evidence present in the trace, so it is natural to ask how often that evidence is there at all. Evidence can arrive through two channels: the injected payload appears in some tool output the monitor sees, or the agent performs an externally-effectful call whose arguments reflect the adversary's goal.

We measured both on the 90 compromised trajectories, and the honest summary is that the observation channel is close to saturated while the action channel is not. Every compromised trajectory contains at least an eight-word contiguous span of the attacker's directive in some tool output it read; 86.7% contain the payload's opening verbatim; 40.0% contain the entire directive verbatim; and 85.6% make an effectful call. The reason the observation

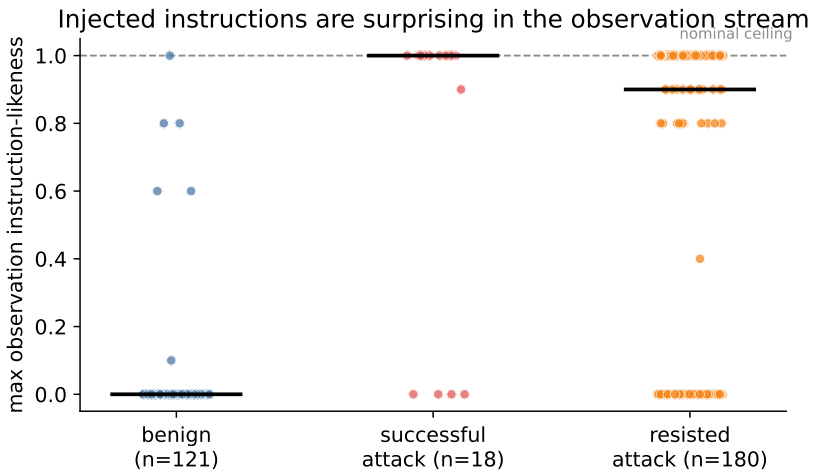

Figure 3: Observation instruction-likeness (maximum over a trajectory) for benign and attacked trajectories, on the DeepSeek-V4-Flash corpus. This is an *attempt*-detection figure, so the benign group includes the 50 τ-bench retail trajectories that the compromise-detection corpora exclude (Section 6.1(viii)). Benign tool output sits near zero at the median, with a minority of benign trajectories reaching the top of the scale—the judge saturation that Section 5 shows costs the signal its operating point. Attacked trajectories sit far above the nominal range whether or not the agent ultimately complied, which is why the signal detects attempts well and compromise less well.

channel is so full is structural: AgentDojo delivers its payload *through a tool output*, and an injection cannot succeed unless the agent read that output.

Remark 5 (Why we do not turn this into a bound)**.** It is tempting to convert these fractions into an information-theoretic ceiling on any trace-only monitor—the fraction of compromises with neither channel would bound every such detector's true-positive rate—and an earlier version of this work did exactly that. We no longer report such a bound, for two reasons that we think are worth stating rather than burying.

First, the quantity is not identified. "The payload is visible" requires a matching criterion, and the answer moves a long way with it: the fraction with neither channel is 0.000 if any eight-word payload span counts, 0.044 if the payload's opening must appear verbatim, and 0.089 if the whole directive must. These payloads share a fixed 32-word wrapper, so the permissive readings largely test for scaffolding, while the strictest reading partly tests how the logging renderer reflowed the text rather than whether evidence was available. We produced three different values for this quantity in the course

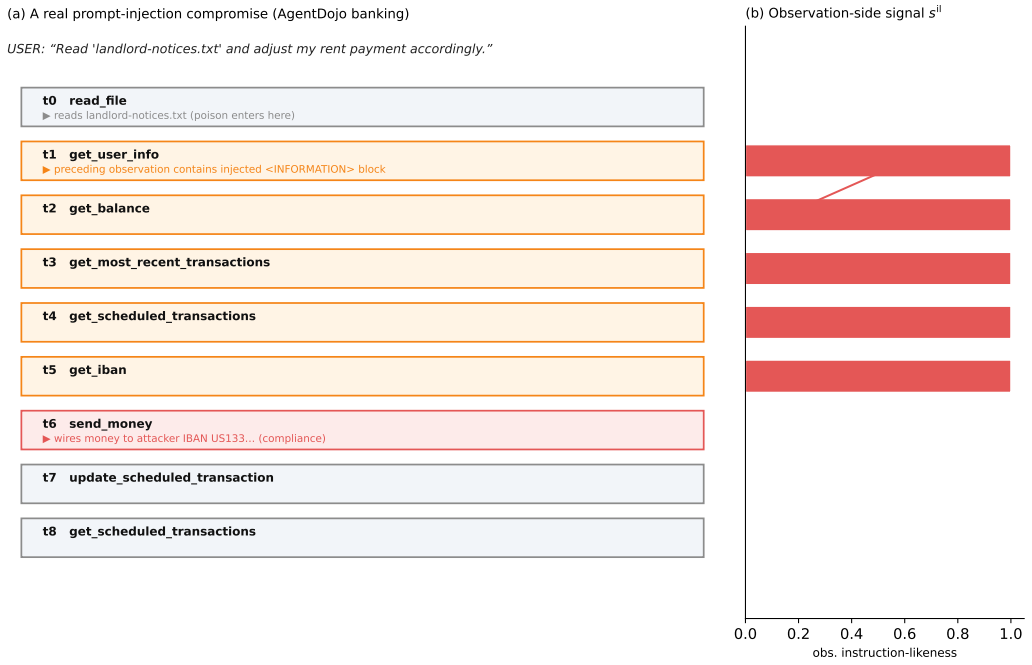

Figure 4: A real prompt-injection compromise, action by action. The user asks the agent to adjust a rent payment; at t_0 it reads a file whose content hides an `<INFORMATION>` block directing it to wire money to the attacker. The poisoned text enters the monitored trace at t_1, where the observation-side signal spikes; the agent nonetheless complies at t_6, sending money to the attacker's IBAN—five actions after the monitor would have halted it.

of this work, and we do not believe any of them is canonical.

Second, the bound such an argument yields is inside the resolution of this corpus. At the strictest criterion it is 0.916, against the 0.898 our method attains—a margin of under two points on 90 positives, which is smaller than the spread we see across calibration draws. A bound that cannot be distinguished from the measurement it is supposed to constrain does no work.

What survives is a descriptive observation and a caution, not a theorem.

The caution is about benchmark construction, and we think it is the more useful half. A corpus that delivers payloads through tool outputs cannot contain a compromise that is invisible to an observation-side monitor, so it cannot test how a monitor behaves when the evidence genuinely is absent. Real deployments admit such cases—an agent persuaded through a channel the monitor never sees, or one whose harmful action is indistinguishable

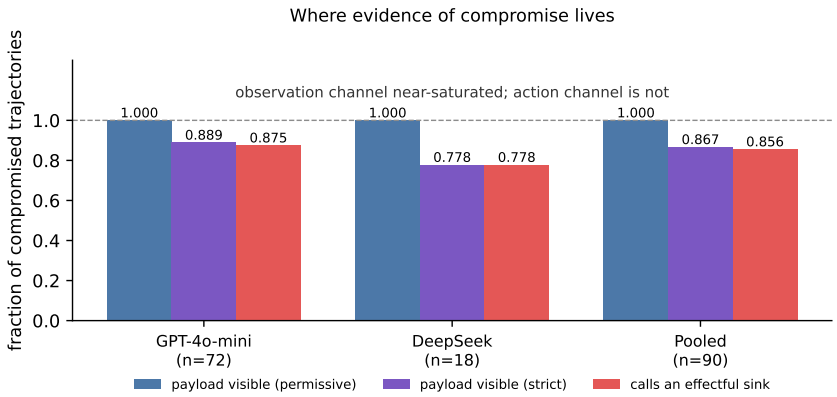

Figure 5: Evidence-channel coverage of the compromised class. "Permissive" counts any eight-word payload span appearing in a tool output; "strict" requires the payload's opening words verbatim. The observation channel is close to saturated on both corpora and the action channel is not, which is why a provenance signal keyed to sinks cannot stand alone. We report these fractions descriptively; Remark 5 explains why we do not convert them into a bound.

from a legitimate one—and no benchmark we are aware of provides them. A defence validated only on corpora of this shape has not been tested against the hard case, and our own results should be read with that limitation in mind (Figure 5): the roughly one in ten compromises we miss are ones we failed to score, not ones that were hidden from us.

5.5. Comparison with a re-implemented baseline

Every comparison so far has been between our own signal subsets, which makes the evaluation a self-ablation. To give it an external anchor we re-implemented a published method and ran it on the same trajectories, with the same splits and the same operating-point rule.

We chose **Task Shield** [5] because it is the closest prior work to our action-side signal: it reframes injection defence as *task alignment*, requiring every agent action to serve the user's stated goal. Our S4 asks a related question, so a reader is entitled to ask whether the fusion adds anything over the published method it most resembles.

What we implemented. We follow the paper's Section 4 and Algorithm 1, using the ACL version's Appendix E prompts. An LLM extracts the set of actionable user task instructions T_u from the user message. For each tool

call e the LLM then scores ContributesTo$(e, t) \in [0, 1]$ against every $t \in T_u$, and the aggregate $S(e) = \sum_t s(e, t)$ decides alignment: e is misaligned iff $S(e) \leq \epsilon$, with $\epsilon = 0$ as in the paper.

What we necessarily changed. Three deviations, all forced by the setting, and all of which we regard as caveats on the comparison rather than improvements to it.

1. Task Shield is a *defence*: on detecting misalignment it injects feedback and blocks the call, changing the trajectory. We score fixed, already-collected traces, so we cannot intervene. We run its alignment check unchanged and read it as a detector—a trajectory is flagged when any of its calls is misaligned. Its published attack-success-rate numbers measure prevention and are therefore not comparable with anything here.
2. The paper's defender is GPT-4o. We use the model that also serves our judge (DeepSeek-V4-Flash, temperature 0), so the comparison isolates the *method* rather than the defender's capability. The paper's own ablation reports Task Shield degrading gracefully with defender capacity and includes a GPT-4o-mini defender, so a weaker defender depresses the absolute numbers; it does not favour either method in the comparison, since both use the same model.
3. Task Shield also checks assistant natural-language content. Our traces contain tool calls and tool outputs but not assistant prose, so only the tool-call checker applies and one of its layers is absent.

Result. Table 6 gives the comparison. Task Shield's published decision rule—flag when any call is misaligned—is a fixed operating point rather than a tunable threshold, so we report it as a single (FPR, TPR) pair alongside its ROC.

At its own operating point the method works: pooled, it catches 0.933 of compromises, which is above the 0.898 SENTRY-Fuse reaches. It does so at a false-alarm rate of 39%. That is the whole of the difference between the two methods, and it is a difference in what they are for. Task Shield was designed as an inline defence, where a false alarm costs an injected clarification and a retry; read as a monitor, where a false alarm costs an analyst's attention, 39% is roughly ten times the budget we hold ourselves to. Alignment with the user's task is a demanding criterion, and benign

agents fail it often—exploratory reads, retries and tool calls that set up later steps all look unmotivated in isolation.

Table 6: Task Shield [5], re-implemented and run on our trajectories under our protocol, against SENTRY-Fuse. Both use the same judge model (DeepSeek-V4-Flash, temperature 0), so the comparison isolates the method rather than the defender. Task Shield's published rule is a fixed operating point, reported as one (FPR, TPR) pair; the AUROC column instead ranks trajectories by their minimum aggregate alignment score. The 5%-target rows are reported for completeness but are not a measure of the method—see the text on quantisation.

Corpus	Detector	AUROC	TPR	FPR
GPT-4o-mini	Task Shield, published rule	—	0.944	30%
	Task Shield, at our target	0.818	0.000	0.0%
	SENTRY-Fuse	**0.981**	**0.914**	2.5%
DeepSeek	Task Shield, published rule	—	0.889	48%
	Task Shield, at our target	0.688	0.000	0.0%
	SENTRY-Fuse	**0.842**	**0.778**	4.0%
Pooled	Task Shield, published rule	—	0.933	39%
	Task Shield, at our target	0.764	0.000	0.0%
	SENTRY-Fuse	**0.953**	**0.898**	3.8%

The zero in the 5% row is an artefact, not a defeat. Held to our own 5% budget Task Shield returns 0.000 recall on every corpus. We want to be explicit that this is not a finding about the method's discriminative power, for the same reason that S4's recall column reads 0.000 in Table 3. The aggregate $S(e)$ is a sum of a handful of judgments each drawn from $\{0, 0.5, 1\}$, so the trajectory score takes only 7 distinct values on the pooled corpus, and 39% of benign trajectories sit at its floor—tied there with 93% of the compromised ones. A 5% false-alarm budget puts the threshold below that tied block, so nothing at all is flagged and both TPR and realised FPR are zero. No threshold on this score can deliver a false-alarm rate between 0 and 39%; the method has no operating point in the range we care about. That is a real limitation for monitoring use, but it is a limitation of resolution rather than of ranking, and reporting the 0.000 without the explanation would misrepresent it.

What the comparison supports. Two things, and not a third. As a ranker Task Shield reaches pooled AUROC = 0.764 against SENTRY-Fuse's 0.953,

and the ordering holds on both corpora separately (0.818 against 0.981, 0.688 against 0.842); on the evidence available to a trace monitor, graded conformal surprisal separates these trajectories better than a binary alignment verdict does. And the quantisation argument above is a genuine obstacle to using an alignment checker at a chosen false-alarm rate. What the comparison does *not* support is any claim that our method is the better *defence*: Task Shield blocks the call it flags, we only raise a flag, and the third of our deviations removes one of its two checking layers. A reader should treat these numbers as locating the two methods relative to each other on our corpus under our reading, not as a verdict on the published system.

5.6. Negative results

A monitor's credibility rests on knowing what it cannot do. We report every component we implemented and rejected.

5.6.1. Signals we implemented and dropped

S1, tool-transition surprise, and S2, argument novelty (near chance). These two behavioural signals were the starting point of this project. Under the three-way split of Section 4.2 they are near-chance: pooled AUROC 0.526 and 0.602 individually and 0.557 together, and on DeepSeek the pair is *below* chance at 0.389 (Table 3). Including them lowers pooled AUROC from 0.953 to 0.948. The reason is visible in the design: both ask whether behaviour is statistically unusual relative to a benign reference, and a compromised agent mostly issues ordinary tool calls in an ordinary order—what is wrong is *which* arguments they carry and *why*, not the shape of the sequence. Two lessons we take from this. First, unusualness is not a proxy for misalignment. Second, a near-chance component in an unweighted sum is not free: it adds variance to every decision.

*Lexical data-flow taint (*AUROC 0.46*).* We scored the fraction of an action's argument tokens traceable to untrusted observations. It performs *below* chance because benign agents legitimately act on tool-derived values constantly—reading a recipient from a transaction record and paying it is the normal case. Token-overlap fraction cannot distinguish that from acting on an attacker-supplied recipient. This failure motivated the conjunctive, sink-restricted form of S5.

*Lexical goal-grounding (*AUROC 0.50*).* Token overlap between the action and the user request is too coarse a proxy for "serves the task". Its semantic replacement, S4, works (0.900 on GPT-4o-mini, 0.781 on DeepSeek)—the idea was right, the implementation was lexical.

Injection-conditioned interaction (no gain). Gating behavioural surprise on a preceding injection produced results identical to the plain sum.

A note on S5. Sink provenance is weak *alone*—pooled AUROC = 0.812 with a true-positive rate of 0.000 at this budget, because it fires only for the subset of attacks that route an identifier into a sensitive sink. Adding it moves pooled recall by only +0.0268 $[-0.0178, +0.1011]$ and AUROC by +0.0004 $[-0.0043, +0.0056]$, both intervals containing zero, which is why it is not in the method.

5.6.2. The anytime-valid machinery did not earn its headline

This work began as an application of e-detectors [26] with PAC-calibrated thresholds [27], on the argument that continuous monitoring demands anytime-valid false-alarm control. We tested that argument directly. Holding the signal fixed and varying only the decision rule, we built 1000-action benign streams by concatenating held-out benign trajectories and swept each rule's knob (Table 7).

Table 7: Long-horizon false alarms on 1000-action benign streams, ten seeds, identical underlying score. All nine configurations we ran are listed, including the two loose fixed thresholds that favour the e-detector's comparison. The e-detector's rows are identical for $\alpha \leq 0.05$ because its PAC threshold saturates at the order statistic our calibration set can supply.

Method	false alarms / 1k	ARL (first alarm)	attack det.
e-detector, $\alpha = 0.2$	45.6	32	0.612
e-detector, $\alpha = 0.05$	23.0	117	0.538
e-detector, $\alpha = 0.01$	23.0	117	0.538
e-detector, $\alpha = 0.002$	23.0	117	0.538
fixed threshold, $q = 0.9$	124.7	19	0.752
fixed threshold, $q = 0.95$	60.9	31	0.658
fixed threshold, $q = 0.99$	19.8	238	0.563
fixed threshold, $q = 0.995$	17.8	334	0.539
fixed threshold, $q = 0.999$	10.2	343	0.524

Two findings stand out, and the first is weaker than we first reported. The e-detector is not simply dominated. At its loosest setting ($\alpha = 0.2$) it detects 0.612 of attacks at 45.6 false alarms per thousand actions, and a fixed threshold matched to roughly that alarm budget ($q = 0.95$: 60.9 per thousand) detects 0.658—better, but at a 34% higher alarm rate. At the tight end the comparison is clearer: $q = 0.999$ gives 10.2 false alarms per thousand at 0.524 detection, against the e-detector's 23.0 at 0.538, so roughly the same detection for less than half the alarms. The honest summary is that a fixed benign quantile traces out a better false-alarm/detection frontier than the e-detector over most of the range, not that it dominates pointwise.

The second finding is the damaging one for the machinery: the e-detector's results are *identical* for $\alpha \in \{0.05, 0.01, 0.002\}$. The PAC threshold saturates at the order-statistic index our calibration set can supply, so the guarantee cannot be tightened below $\alpha = 0.2$, where it promises only ARL ≥ 5—a vacuous promise for a monitor. A cross-agent transfer test showed no benefit either. The anytime-valid apparatus this project began from therefore did not earn its place, and we removed it from the method rather than keep it for the theory it advertises. What would change this is a calibration set one to two orders of magnitude larger, at which point the order statistic exists and the guarantee becomes both valid and non-vacuous. We consider this a useful negative result for a community currently enthusiastic about anytime-valid monitoring.

6. Discussion and Conclusion

What the measurement changes. We set out expecting to find that trace-only detection is limited by the evidence available in the trace, and to report how close to that limit a good fusion rule gets. Neither half survived. The observation channel is close to saturated on AgentDojo, so the limit is not what constrains a detector here; and the limit itself turned out not to be identifiable from this corpus, which is why Remark 5 reports fractions rather than a bound. The framing that trace-only injection detection is a modelling problem is, on this corpus, the correct one—and our contribution is accordingly a modelling contribution, not an information-theoretic one. Where the evidence framing does bite is benchmark construction: a corpus that delivers payloads through tool outputs cannot tell us how a monitor behaves when the evidence really is absent.

Why several channels rather than one. S3 and S4 fail on largely different cases. S3 misses compromises whose payload is present but unremarkable in phrasing; S4 misses compromises whose harmful action looks superficially plausible given the request; S5 fires only when an identifier reaches a sink, but does so with high precision when it does. The evidence for complementarity is that the fusion beats its best component by more than the components' individual strengths would suggest: pooled, S3 alone gives 0.937 AUROC and 0.709 recall while the pair gives 0.953 and 0.898, even though S4 alone reaches no recall at all at this budget. The design principle we draw is narrower than "prefer diverse signals": prefer signals that are diverse *and individually better than chance*, since Section 5.6 shows a diverse but near-chance component actively hurts an unweighted sum.

Agent robustness is a confound. GPT-4o-mini was compromised five times more often than DeepSeek-V4-Flash on identical attacks, and DeepSeek resisted an entire attack family. A defence evaluated on one agent is partly measuring that agent. We recommend reporting the agent's unprotected compromise rate alongside any detector's TPR, as Table 1 does.

Cost. S3 and S4 each require one small-model call per distinct observation and per distinct action, cached by content hash; across the 584 trajectories these caches cover (the 393 attacked plus the 191 benign used for calibration) this is 2,010 calls in total, and zero at re-evaluation time. There is no GPU, no access to model internals, and no agent re-execution, in contrast to re-execution defences [18].

Deployment and interpretability. Because (9) is additive over interpretable components, an alarm comes with an attribution: the dominant ψ_k names the reason. An operator can route "injected instruction detected" differently from "action unjustified given the user's request", and can tune α against a false-alarm budget.

6.1. Limitations

(i) Adaptive adversaries. Our attacks are static templates. An adversary aware of SENTRY-Fuse would phrase injections to minimise instruction-likeness *and* route effects through non-sink tools *and* keep actions superficially task-plausible. Requiring simultaneous evasion of signals that fail on different inputs raises the bar, but we have not measured by how much, and Nasr et al. [8] caution against assuming such bars hold.

(ii) Corpus scale and breadth. 90 compromised trajectories, two agents, two attack families, one benchmark family. The DeepSeek compromised class has only 18 members, and on that corpus fusion does not improve ranking quality at all: AUROC is 0.842 against the single signal's 0.850. It helps recall there ($0.467 \to 0.778$) but that is a smaller claim on 18 positives. We report both corpora separately and pooled rather than presenting the more favourable one. A partial τ-bench airline collection (11 of 50 tasks) is present in the released data but is used in no evaluation, being too small to add a meaningful benign stratum.

(iii) Judge dependence and a judge/agent overlap. S3 and S4 depend on a small LLM—DeepSeek-V4-Flash at temperature 0—whose behaviour may drift with model updates; the cached-score design makes a given evaluation reproducible but does not make the judge itself stable over time. Note also that this is the *same* model as one of the two monitored agents, so on the DeepSeek corpus the judge and the agent share a model family. We did not control for this, and it is a candidate explanation for part of the gap between the two corpora.

(iv) The monitor is blind to the agent's own words. The trace we score contains tool calls and tool outputs, not the agent's natural-language messages. A compromise that discloses information in prose without touching a tool would be invisible to both signals. Because this corpus delivers every payload through a tool output, almost no such case appears among our positives—which is a limitation of the corpus rather than evidence that the case is rare.

(v) Quantisation of the conformal transform. Proposition 3(iii) shows ψ saturates above the calibration maximum, so with small calibration sets it can lose discrimination: at the $n = 47$ used here we measure $1.000 \to 0.990$ on separable synthetic data, and $1.000 \to 0.924$ at $n = 5$. It can also, in adverse configurations, take an above-chance signal below chance. Larger calibration splits reduce the loss, at the cost of fewer held-out negatives and hence a coarser attainable FPR grid—a trade-off this corpus is too small to escape.

(vi) No information-theoretic guarantee. We report no bound on what a trace-only monitor can achieve. Remark 5 explains why: the fraction of compromises leaving no trace evidence is not identified on this corpus, and at the strictest criterion the bound it would give is violated by our own measured true-positive rate. Readers should therefore not treat our numbers as close to, or far from, any limit—we do not know where the limit is.

(vii) Compromise-detection transfer across attack families is untested. This is the most consequential gap in our evaluation, and it is structural rather than presentational. We have InjecAgent data for one agent only, and that agent resisted the family completely (0/42), so its compromise-positive class is empty. Every compromise-detection number in this paper is therefore measured on `important_instructions` attacks alone; the transfer evidence in Table 4 concerns *attempt* detection, a different and easier objective. Closing this requires collecting InjecAgent attacks against an agent that actually succumbs to them—a data-collection task, not an analysis one—and until that is done a reader should assume our compromise-side results may be specific to one attack template.

(viii) A single benchmark family, and no τ-bench in the compromise corpora. The compromise-detection corpora are AgentDojo only. An earlier version pooled in τ-bench retail trajectories as extra benign data; we removed them, because they mix tool vocabularies in a way that strains the exchangeability Proposition 2 assumes and—worse—they were loaded without their message list, so S5 was silently fixed at zero for all 50 rather than measured. That inflated S5's apparent standalone recall from 0.000 to 0.220. τ-bench is still used for attempt detection, where S5 plays no part. The cost of removing them is a smaller benign reference (141 pooled rather than 191) and a coarser attainable false-positive grid.

6.2. Future work

Monitor the response channel. Extending the trace to the agent's natural-language messages would let the monitor catch information-disclosure compromises that never touch a tool—the case this corpus barely exhibits, since its payloads arrive through tool outputs.

Build a corpus with hidden compromises. The more useful consequence of Section 5.4 is a benchmark requirement. A corpus in which some successful injections leave no trace evidence—delivered out of band, or achieving their goal through actions indistinguishable from legitimate ones—is what would let the community measure how monitors behave when information genuinely runs out. No existing benchmark we are aware of provides this.

Score content at the retrieval boundary. A monitor placed where documents *enter* the environment, rather than where the agent surfaces them, would see payloads the agent never reads aloud.

Structured provenance. S5 is a coarse approximation of the program-dependence analysis that Wang et al. [3] perform. A faithful data-flow implementation, fused rather than used standalone, is a natural strengthening, and re-implementing it on this corpus would extend the baseline comparison of Section 5.5 to the strongest reported trace-only detector, which Section 4.3 explains we cannot currently make.

Adaptive evaluation. Attacks optimised jointly against both retained signals are the decisive robustness test.

6.3. Conclusion

We set out to build a better trace-only monitor for prompt-injection compromise in LLM agents, and to establish where the information in a trace runs out. SENTRY-Fuse converts heterogeneous trace signals into conformal tail surprisals and fuses them by addition; the construction is scale-free, non-decreasing in every component and free of tuned weights, and we proved that the natural alternative of subtracting the benign training maximum is many-to-one and, for any attack class lying below that maximum, drives AUROC to at most one half—as it did, measurably, in an earlier and unpublished draft of this work, where it cost 0.25 AUROC on the composite score and produced an explanation of the failure that was confidently wrong.

Fusing one observation-side signal with one action-side audit attains AUROC $= 0.981 \pm 0.009$ and a true-positive rate of 0.914 ± 0.040 at a realised 2.5% false-positive rate on the GPT-4o-mini corpus, and 0.953 ± 0.005 / 0.898 ± 0.020 pooled over 90 compromised trajectories from two agents. We are deliberately careful about what this establishes. Resampling trajectories, the recall gain over the instruction signal alone is $+0.1706$ $[+0.0033, +0.3533]$ and the AUROC gain $+0.0163$ $[-0.0006, +0.0351]$; the latter interval only marginally excludes no effect, and 90 positives cannot settle it. The effect we would defend is different in kind: the single signal returns *zero* recall in 2 of 10 calibration draws, because a saturating judge and a discrete operating point conspire to put the threshold at the ceiling, and fusion removes that failure entirely. A detector that silently returns nothing on a fifth of calibration draws cannot be deployed at a stated operating point; that is the practical contribution. These figures sit in the same regime as the strongest reported trace-only detector, but we claim no comparison with it: the corpora, agents and operating points differ, and its program-dependence analysis is not something we re-implemented. The external anchor we do

provide is a re-implementation of Task Shield run on our own trajectories under our own protocol (Section 5.5).

Our expectation going in was that an information limit would explain the residual error, and an earlier version of this work reported one. We withdraw it. The fraction of compromises leaving no trace evidence is not identified on this corpus—three defensible matching criteria give 0.000, 0.044 and 0.089—and at the strictest of them the implied bound falls below the true-positive rate we measure, which means the bound is not usable rather than that we have beaten it. We report the underlying measurement descriptively instead: the observation channel is close to saturated because AgentDojo delivers its payloads through tool outputs a successful injection must have read. The consequence is about benchmarks rather than about monitors. A corpus of this shape cannot contain a compromise that is invisible to a trace monitor, so it cannot test the case that matters most, and the question of whether the compromises we miss lacked evidence or were merely mis-scored is not answerable on this corpus.

Alongside this we document a label-polarity trap in a standard harness that silently inverted our own conclusions, and we report in full the components we rejected: two behavioural signals and one provenance signal that were part of earlier versions of this method, two lexical signals that failed outright, and the anytime-valid machinery this project began from, which a hand-picked threshold outperformed. We think the negative results are the more transferable half of the paper.

Data availability

The agent trajectories, the cached judge scores and the analysis code that regenerate every number, table and figure in this paper are publicly available at
`https://github.com/vinhqdang/SENTRY-Anytime-Valid-E-Process-Monitoring-of-LLM-Agent-Action-Streams`
Appendix A describes the contents and the reproduction procedure.

Declaration of competing interest

The author declares no competing interests. No external funding was received.

Appendix A. The released artefact

The evaluation is released in full so that the results can be checked rather than taken on trust. Three properties are worth stating, because together they determine what a reader can verify.

The corpora are released, not just the code. The 191 benign and 393 attacked agent trajectories described in Section 4 are committed as raw logs, with the harness metadata from which the compromise labels of Section 4.4 are derived. A reader who disagrees with our labelling can re-derive it.

The model-judge scores are cached. Both adopted signals query a language model: S3 scores each distinct observation and S4 audits each distinct tool call. Every such judgment is committed, keyed by a hash of its input. The evaluation therefore reproduces without an API key, without network access and without expenditure, and—because language-model sampling is not bit-reproducible—it reproduces to the digit rather than approximately. The same holds for the Task Shield re-implementation of Section 5.5, whose alignment judgments are cached the same way.

The reported values are generated, not transcribed. Every quantity quoted in the prose and in the tables is emitted as a macro from the results files, so the text and the tables cannot disagree with each other or with a re-run. A verification pass re-derives each value independently and fails if any of them departs from the manuscript, which is how the numbers in this paper are checked.

The repository also retains the components this paper reports as negative results—the two behavioural signals, the sink-provenance signal, the excess-over-maximum normalisation and the anytime-valid detector of Section 5.6—so that the rejections in Section 5.6 can be reproduced alongside the adopted method. Superseded scripts are retained but marked as such, to prevent them being mistaken for the current method.

References

[1] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents, in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 10471–10506. URL: https://aclanthology.org/2024.findings-acl.624/. doi:10.18653/v1/2024.findings-acl.624.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track, 2024. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[3] P. Wang, Y. Liu, Y. Lu, Y. Cai, H. Chen, Q. Yang, J. Zhang, J. Hong, Y. Wu, AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection, arXiv preprint arXiv:2508.01249v1 (2025). URL: https://arxiv.org/abs/2508.01249v1. doi:10.48550/arXiv.2508.01249, version 1. The 95.75%/3.66% operating point cited here appears in v1 and is not present in later versions.

[4] Y. Liu, Y. Jia, J. Jia, D. Song, N. Z. Gong, DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks, in: 2025 IEEE Symposium on Security and Privacy (SP), IEEE, San Francisco, CA, USA, 2025, pp. 2190–2208. URL: https://doi.org/10.1109/SP61157.2025.00250. doi:10.1109/SP61157.2025.00250.

[5] F. Jia, T. Wu, X. Qin, A. Squicciarini, The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 29680–29697. URL: https://aclanthology.org/2025.acl-long.1435/. doi:10.18653/v1/2025.acl-long.1435.

[6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), Association for Computing Machinery, Copenhagen, Denmark, 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[7] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, A. Beutel, The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instruc-

tions, arXiv preprint arXiv:2404.13208 (2024). URL: https://arxiv.org/abs/2404.13208. doi:10.48550/arXiv.2404.13208.

[8] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, F. Tramèr, The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections, arXiv preprint arXiv:2510.09023 (2025). URL: https://arxiv.org/abs/2510.09023. doi:10.48550/arXiv.2510.09023.

[9] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025. URL: https://iclr.cc/virtual/2025/poster/28170.

[10] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, G. Neubig, WebArena: A Realistic Web Environment for Building Autonomous Agents, in: The Twelfth International Conference on Learning Representations (ICLR), 2024. URL: https://openreview.net/forum?id=oKn9c6ytLx.

[11] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, E. Winsor, J. Wynne, Y. Gal, X. Davies, AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025. URL: https://openreview.net/forum?id=AC5n7xHuR1.

[12] M. Q. Li, B. C. M. Fung, B. Li, H. Ismail, F. Iqbal, Taxonomy and Consistency Analysis of Safety Benchmarks for AI Agents, arXiv preprint arXiv:2605.16282 (2026). URL: https://arxiv.org/abs/2605.16282. doi:10.48550/arXiv.2605.16282.

[13] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, E. Kıcıman, Defending Against Indirect Prompt Injection Attacks With Spotlighting, in: Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024), volume 3920 of *CEUR Workshop Proceedings*, Arlington, Virginia, USA, 2024, pp. 48–62. URL: https://ceur-ws.org/Vol-3920/paper3.pdf.

[14] S. Chen, J. Piet, C. Sitawarin, D. Wagner, StruQ: Defending Against Prompt Injection with Structured Queries, in: 34th USENIX Security Symposium (USENIX Security '25), USENIX Association, 2025. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe.

[15] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, C. Guo, SecAlign: Defending Against Prompt Injection with Preference Optimization, in: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), Association for Computing Machinery, Taipei, Taiwan, 2025, pp. 2833–2847. doi:10.1145/3719027.3744836.

[16] D. Das, L. Beurer-Kellner, M. Fischer, M. Baader, CommandSans: Securing AI Agents with Surgical Precision Prompt Sanitization, arXiv preprint arXiv:2510.08829 (2025). URL: https://arxiv.org/abs/2510.08829. doi:10.48550/arXiv.2510.08829.

[17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating Prompt Injections by Design, arXiv preprint arXiv:2503.18813 (2025). URL: https://arxiv.org/abs/2503.18813. doi:10.48550/arXiv.2503.18813.

[18] K. Zhu, X. Yang, J. Wang, W. Guo, W. Y. Wang, MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), volume 267 of *Proceedings of Machine Learning Research*, PMLR, Vancouver, Canada, 2025, pp. 80310–80329. URL: https://proceedings.mlr.press/v267/zhu25z.html.

[19] I. Belenky, J. Itria, S. Johns, Cheap Reward Hacking Detection, arXiv preprint arXiv:2606.08893 (2026). URL: https://arxiv.org/abs/2606.08893. doi:10.48550/arXiv.2606.08893.

[20] K. Thaman, Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use, arXiv preprint arXiv:2605.02964 (2026). URL: https://arxiv.org/abs/2605.02964. doi:10.48550/arXiv.2605.02964.

[21] J. Opoku, D. Banahene, ToolChain-CRC: Conformal Risk Control for Agentic AI Under Retrieval and Tool-Use Drift, arXiv preprint arXiv:2606.18467 (2026). URL: https://arxiv.org/abs/2606.18467. doi:10.48550/arXiv.2606.18467.

[22] V. Vovk, A. Gammerman, G. Shafer, Algorithmic Learning in a Random World, 2nd ed., Springer, Cham, Switzerland, 2022. doi:10.1007/978-3-031-06649-8.

[23] V. Vovk, R. Wang, E-values: Calibration, combination and applications, The Annals of Statistics 49 (2021) 1736–1754. doi:10.1214/20-aos2020.

[24] A. Ramdas, P. Grünwald, V. Vovk, G. Shafer, Game-Theoretic Statistics and Safe Anytime-Valid Inference, Statistical Science 38 (2023) 576–601. doi:10.1214/23-sts894.

[25] P. Grünwald, R. de Heide, W. M. Koolen, Safe testing, Journal of the Royal Statistical Society Series B: Statistical Methodology 86 (2024) 1091–1128. doi:10.1093/jrsssb/qkae011.

[26] J. Shin, A. Ramdas, A. Rinaldo, E-detectors: A Nonparametric Framework for Sequential Change Detection, The New England Journal of Statistics in Data Science 2 (2024) 229–260. doi:10.51387/23-nejsds51.

[27] S. Sadhuka, D. Prinster, C. Fannjiang, G. Scalia, B. Berger, A. Regev, H. Wang, E-valuator: Reliable Agent Verifiers with Sequential Hypothesis Testing, arXiv preprint arXiv:2512.03109 (2025). URL: https://arxiv.org/abs/2512.03109. doi:10.48550/arXiv.2512.03109.

[28] I. Waudby-Smith, A. Ramdas, Estimating means of bounded random variables by betting, Journal of the Royal Statistical Society Series B: Statistical Methodology 86 (2024) 1–27. doi:10.1093/jrsssb/qkad009.

[29] E. S. Page, Continuous Inspection Schemes, Biometrika 41 (1954) 100–115. doi:10.1093/biomet/41.1-2.100.

[30] G. Lorden, Procedures for Reacting to a Change in Distribution, The Annals of Mathematical Statistics 42 (1971) 1897–1908. doi:10.1214/aoms/1177693055.

[31] Y. Xie, Sequential statistical inference for Large Language Models: Representation, validity, and monitoring, arXiv preprint arXiv:2606.07624 (2026). URL: https://arxiv.org/abs/2606.07624. doi:10.48550/arXiv.2606.07624, Prepared for an invited discussion in *The American Statistician*.

[32] Y. Li, Who Drifted: the System or the Judge? Anytime-Valid Attribution in LLM Evaluation Pipelines, arXiv preprint arXiv:2606.15474 (2026). URL: https://arxiv.org/abs/2606.15474. doi:10.48550/arXiv.2606.15474.

[33] E. Gauthier, F. Bach, M. I. Jordan, Anytime Detection of Strategic Deviations in Multi-Agent Systems, arXiv preprint arXiv:2601.05427 (2026). URL: https://arxiv.org/abs/2601.05427. doi:10.48550/arXiv.2601.05427.

[34] N. Feng, Y. Sui, S. Hou, G. Wu, J. C. Cresswell, Conformal Agent Error Attribution, arXiv preprint arXiv:2605.06788 (2026). URL: https://arxiv.org/abs/2605.06788. doi:10.48550/arXiv.2605.06788.